

Angular – Best Practices

v1.2.0

26-October-2026

CO-CREATE TO OUTPERFORM | WNS

Prepared By:	Malay Nirav
Reviewed By:	Nikitha Amaresh
Approved By:	Vivek Kumar

Version	Date	Change Mode (C/A/M/D)	Description of change	Prepared/ Revised by	Reviewed By	Approved By
1.0.0	12/02/2024	C	First version	Malay Nirav	Nikitha Amaresh	Vivek Kumar
1.1.0	28/06/2024	M	2.20 Angular CSS Guidelines	Malay Nirav	Nikitha Amaresh	Vivek Kumar
1.2.0	26/10/2026	A	2.20.7 CLS; 2.21 Zoneless; 2.22 Rendering Strategy	Pending	Pending	Pending

C – Created, A – Added, M – Modified, D – Deleted

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Contents

1. **Introduction** 1

2. **Coding Guidelines** 3

3. **Application Pre-requisite** 13

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

1. Introduction

Angular is an application design framework and development platform for creating efficient and sophisticated single-page apps.

The following diagram shows how these basic pieces are related.

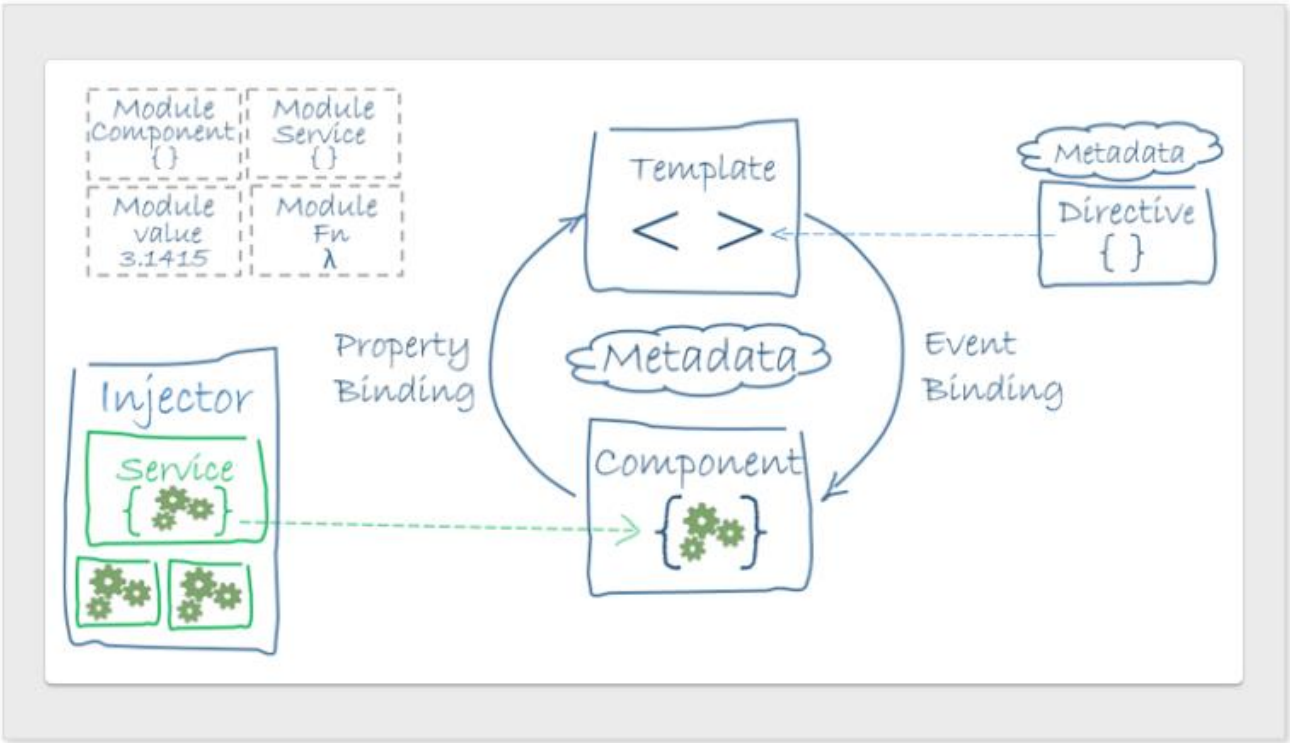


Image Courtesy: https://angular.io/generated/images/guide/architecture/overview2.png

The above architecture diagram identifies the following main building blocks

Modules	NgModules help organize an application into cohesive blocks of functionality.
Components	A component is a directive with a template that allows building blocks of a UI in an Angular application. Components will always have a template, a selector, and may or may not have a separate style. Components are declared using @Component.
Templates	A template is a form of HTML that tells Angular how to render the component.
Metadata	Metadata tells Angular how to process a class
Data binding	Binding between data and the view

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Directives

Angular templates are *dynamic*. When Angular renders them, it transforms the DOM according to the instructions given by **directives**.

Two other kinds of directives exist:

- structural : Alters the layout e.g ngFor, ngIf
- attribute: Alters the appearance or behavior of an existing element e.g ngModel, ngClass, ngSwitch etc

Services

Services are a great way to share information among classes that don't know each other.They are typescript classes with @injectible decorator

Dependency Injection

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

2. Coding Guidelines

Following best practices in Angular would enable us to deliver a performant web application

1. Make use of Angular CLI

The Angular CLI is a command-line interface tool that you use to initialize, develop, scaffold, and maintain Angular applications directly from a command shell. It makes it easier for developers to create the app structure by saving a lot of time.

```
npm install -g @angular/cli
```

Few essential commands to use with Angular CLI

ng new	Create an default working application
ng generate (ng g)	Used to generate component, services , routes and pipes with tests
ng serve	Running app in local development server
ng test	For running unit test on your code
ng lint	Helps to lint your code
ng build	To bundle your code

2. Angular Coding Styles

Code quality and coding style play a pivotal role in angular development. Developers needs to stick to the Angular coding guidelines by following the Angular coding styles.

Here are some Angular coding best practices to follow in order to ensure that your angular development project adheres to the right code standards.

- Files should be limited to 400 lines of code.
- Create short functions with no more than 75 lines of code.
- All symbols should have the same name.
- Declare it with ‘const’ if the variables’ values aren’t changing.
- Separate terms in the descriptive name with dashes, and separate the descriptive name from the type with dots.
- Property and method names should always be written in lower camel case.
- Always leave an empty line between imports and modules
- Refer Angular Style Guide for naming conventions

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

3. **Document without fail**

Documentation is very important for code maintenance overtime and when working as a team. It gives a better understanding of the code. It helps in standard way of documentation. Use tsdoc or compdoc to generate documentation

```
/**
 * @remarks
 * Description on what the function is doing.
 *
 * @param x - The first input number
 * @param y - The second input number
 * @returns information on what is being returned
 */
```

Usage

```
# Install compodoc with npm
$ npm install -g @compodoc/compodoc
# Run compodoc in your project (generated with Angular CLI for example) and
serve it
$ compodoc -p src/tsconfig.app.json -s

# Install typedoc with npm
npm install --save-dev typedoc
# Run compodoc in your project (generated with Angular CLI for example) and
serve it
typedoc --out docs /src
```

4. **Single Responsibility Principle**

It is desirable to create separate ts files for components, templates, styles, and services rather than collecting them all together under a single ts file. All the files are responsible for creating a single functionality and if you do that you will be able to create clean, readable, and maintainable code.

Don't create more than one instance of component, service or directive inside the same file

5. **Component Creation**

One of the best practice is to break larger components into smaller components to the extent that each component performs atomic task. This approach aids in ease of debugging and code maintainability

6. **Utilizing 'trackBy' along 'ngFor'**

Using only *ngFor directive without trackBy function in an angular application will remove all the DOM elements and then it will recreate the DOM elements again in the DOM tree. So even when the same data is getting used, it'll slow the application performance when there is a lot of data. That is why it is

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

good to use trackBy along with *ngFor directive.
In component.ts file:

```
trackByCode(index: number, book: any): string {
        return book.code;
}
```

In component.html file:

```
<div class="col-sm-12" *ngFor='let book of books; trackBy:trackByCode'></div>
```

7. Optimizing Angular Files and folder

It's very important to organize the project folder structure for ease of maintainability and reuse

```
|--app
       |--modules
           |--landing
               |--[+]components
               |--landing-routing.module.ts
               |--landing.module.ts
       |--core
           |--[+]header
           |--[+]footer
           |--[+]sidebar
           |--[+]guard
           |--[+]interceptors
           |--core.module.ts
       |--shared
           |--[+]coponents
           |--[+]directives
           |--[+]pipes
           |--[+]services
       |--app-routing.module.ts
       |--app.module.ts
  |--assets
       |--scss
       |--config
```

Root Module: For starting the angular application, call as App module and it loads all root components and other modules.

Core Module: Main components that are used once are created here. It should be initiated only once from the root module

Shared Module: All common components, services or pipes needs to be created under shared module

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Feature Modules: It's good to organize the different features in your application as feature modules , it keeps the code organized

Naming Convention

Class Naming	UpperCamelCase e.g. AuthService
File Naming	File name should convey the purpose. Be consistent with naming patterns. File names needs to be descriptive, if required use "-" to separate the file e.g hero.component.ts
Selectors / Directives	Use lower camel case
Project name	Use lowercase without spaces

8. **Build Reusable component**

Do you need a ui component or function to be used at multiple places in the project? Then this is the right time to create the reusable component.

- This would bring consistency to your application
- Reduce bug fixing time
- Handy while redesigning the component and save time

9. **Lazy Loading**

By default, NgModules are eagerly loaded, which means that as soon as the application loads, so do all the NgModules, whether or not they are immediately necessary. For large applications with lots of routes, consider lazy loading —a design pattern that loads NgModules as needed. Lazy loading helps keep initial bundle sizes smaller, which in turn helps decrease load times.

```
const routes: Routes = [
  {
  # Without lazy loading
  path:'not-lazy-loaded', component: NotLazyLoadedComponet }
  # With lazy loading
    path: 'lazy-loading',
    loadChildren: () => import('./modules/lazy/lazy.module').then(m =>
m.LazyModule)
  }
];
```

10. **Avoid logic in component**

Mixing logic with components creates complexity and untidy code. Testing off UI differs from the logic, having logic in services would improve code reusability and quality

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

11. Prevent memory leaks

Memory leaks gradually degrades the performance of the application. Following are some of the best practices and suggestion to overcome the memory leaks

- Use of async pipes

  Use observables or asyn pipe and promise to avoid memory leak. Avoid subscribing observable in component and binding it in view

```
<ul>
  <li *ngFor="let item of data | async";{{item.item}}</li>
</ul>
```

- Using ‘take(1)’

  Ensure that you receive data only once

```
$data.pipe(take(1)).subscribe(res = {this.list = res;})
```

- Using ‘takeUntil’

  It is used when you want to monitor the two observables and dispose of the subscription after observable emit the value or get completed

```
this.authService.GetBookList()
      .pipe(takeUntil(this.ngUnsubscribe))
      .subscribe(res => {this.bookList = res;});
ngOnDestroy() {
  this.ngUnsubscribe.next();
  this.ngUnsubscribe.complete();
}
```

12. Memory issues

On almost every single Angular project after a certain size, you’ll start running into these errors:

- An unhandled exception occurred: Call retries were exceed
- Javascript Heap Out of Memory

Solution is to set the node heap size as follows

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

```
"scripts": {
  "ng": "ng",
  "start": "ng serve",
  "build": "ng build",
  "test": "ng test",
  "lint": "ng lint",
  "e2e": "ng e2e",
  "build-prod": "node --max_old_space_size=8000
./node_modules/@angular/cli/bin/ng build --prod",
}
```

13. Cache API calls

Caching API calls improves performance and saves memory by limiting server requests to fetch redundant information. Some API responses may not change frequently so we can put some caching mechanism and can store those values from the API. When the same API request is made, we can get a response from the cache. Thus, it speeds up the application and also makes sure that we are not downloading the same information frequently.

14. Declare Safe Data Types

You will initially have to start by confirming the types of data and the set of values that it holds. After that only the possible values will be accepted by the variable. Instead of declaring a variable of a particular type, we can declare it as a list of possible values or a different type.

```
type WeekEnd = "Saturday" | "Sunday";

const day1: WeekEnd = "Saturday";
const day2: WeekEnd = "Sunday";
const day3: WeekEnd = "Monday";

// The last line will give this error: Type ‘"Monday"’ is not assignable to
type 'WeekEnd'.
```

15. Write unit test

Unit test cases helps in improving code quality and reducing regression bugs. Its very useful when multiple developers works on the same application.

Karma: It is the default test runner for Angular. It includes the option to test the code on several browsers and even devices to ensure the smooth operation of the code on the supported platforms.

Jasmine: It is BDD (Behavior-Driven Development) framework for writing unit test cases

16. Configuration

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

Move all the configurable values to config file. To support multi environment deployment with one build move the configuration from enviornment.ts file to config file which could be modified post the build is created

17. **Build Setup**

- Always test your code by building it in production mode before committing to the repository. Suppressed warning in development environment will be a road block in production environment.
- Create Docker file for deployment
- Create Pipelines for automated build to test the application in the development server

18. **Security**

Follow the security best practices

- Run npm audit to check for vulnerabilities
- Avoid customizing angular core library
- Avoid server side template engine
- Never use template generated by concatenating user input - Never concatenate any potentially user provided input as a string to a template.
- Never use native DOM API to interact with HTML elements on the page - Avoid direct DOM manipulation and use Angular template mechanisms, and Angular's own APIs to manipulate the DOM instead.Instead of JQuery use ElementRef API
- THE "ANGULAR WAY" safeguards you from XSS - use interpolation ({{ }}) to safely encode potentially dangerous characters and escape untrusted HTML or CSS expressions within a template expression
- Use innerhtml with caution - If you must dynamically add HTML to a component, bind its generation to [innerHTML]. This ensures data will be interpreted as HTML in its context and sanitized, removing all unsafe tags and hence preventing it from executing any malicious cross-site scripting code. Notice that the action of *sanitizing* is not the same as *encoding*.

19. **Things to avoid**

- Falling back to JQuery implementation – Avoid using JQuery for DOM manipulations
- Avoiding the use of 'any' type can potentially lower the number of unexpected issues. Also, not using any type in our application will make the refactoring uncomplicated. The problems can be restricted by typing the specific variables and keeping others away.
- Avoid making custom changes to angular core libraries, this may affect the security patches being applied
- Avoid inline css

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

20 Component-Level Styles

- Use Angular's component-specific styles to encapsulate styles within components, preventing global scope pollution.
- Utilize the `styles` and `styleUrls` properties in the component metadata to link styles

20.1 Modular CSS:

- **Create** independent, reusable CSS files for each component to enhance maintainability and prevent conflicts.
- **Organize** CSS within components using `:host, :host-context` pseudo-classes to encapsulate styles.
- **Consider** using tools like `ng-build` to optimize unused CSS and avoid bloating.

20.2 Naming Conventions:

- **Adopt** consistent naming conventions for classes using BEM, SUIT, or other established methods.
- **Prefix** class names with the component name to distinguish and prevent conflicts between components.
- **Use** descriptive names, avoiding abbreviations, to improve readability and maintainability.

20.3 Preprocessing:

- **Leverage** CSS preprocessors like Sass/SCSS and Less to enjoy variables, nesting, mixins, and other features for cleaner and more manageable code.
- **Utilize** features like mixins to write reusable code blocks and maintain consistency across the project.
- **Implement** variables for colors, sizes, and other frequently used values for global consistency and easier updates.

20.4 Styling Techniques:

- **Leverage** Flexbox, Grid, and media queries for flexible and responsive layouts that adapt across various screen sizes.
- **Employ** pseudo-classes and attributes to target specific elements within components and enhance interactivity.
- **Utilize** animations and transitions to create smooth visual effects and enhance user experience.

20.5 Testing and Auditing:

- **Integrate** CSS testing frameworks like Protractor to automate CSS testing and ensure component styles work as intended.
- **Regularly** run CSS linters to identify and rectify potential syntax errors, rule conflicts, and stylistic inconsistencies.
- **Utilize** CSS auditing tools to analyze code complexity, performance, and identify opportunities for optimization.

20.6 Additional Recommendations:

- **Follow** Angular Style Guide recommendations for CSS development.
- **Use** Angular CLI commands like `ng serve` and `ng build` to optimize CSS during development and production builds.
- **Consider** using pre-built libraries and frameworks like Bootstrap, Material, or PrimeNG to accelerate development and ensure standardized styles.
- **Take** advantage of community resources like StackOverflow, Angular forums, and tutorials for assistance and best practices sharing.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

20.7 Cumulative Layout Shift (CLS) Optimization:

Unexpected layout shifts reduce the visual stability of a page and degrade the user experience, for example when content moves while a user is reading or about to click. Follow the practices below to keep layout shifts to a minimum.

- **Define** `width` and `height` attributes on images and videos, or use the CSS `aspect-ratio` property, to reserve space before they load.
- **Reserve** appropriate space for dynamically loaded content, advertisements, embedded content, and loading placeholders, for example with `min-height` or `aspect-ratio`.
- **Avoid** introducing content above existing visible content in ways that cause unexpected shifts, such as banners that appear after the page has rendered. Reserve the space in advance or overlay the content instead.
- **Prefer** CSS `transform` and `opacity` for animations where appropriate, rather than properties that trigger layout changes, such as `top` and `left`.
- **Measure** CLS using tools such as Lighthouse and PageSpeed Insights. Lab tools typically measure the initial page load only, so also review field data for shifts that occur later.
- **Target** a CLS score of 0.1 or less at the 75th percentile of page visits, following Google's Core Web Vitals guidance.
- **Refer** https://web.dev/articles/optimize-cls for further guidance.

21. Zoneless Change Detection

Zoneless change detection allows Angular to run without depending on Zone.js to trigger change detection. Change detection still runs; Angular is notified through its own APIs, such as Signals, instead of through patched browser APIs.

- New applications must follow the defaults of the approved Angular version.
- For Angular 21 and later, use zoneless change detection by default unless a documented compatibility requirement justifies an alternative. Verify that `provideZoneChangeDetection` is not used to override the default configuration.
- In Angular 20, zoneless change detection is enabled explicitly by adding `provideZonelessChangeDetection()` to the application providers.

```
// Angular 20 only. Zoneless is the default from Angular 21.
bootstrapApplication(AppComponent, {
  providers: [provideZonelessChangeDetection()],
});
```

- Prefer supported change-notification mechanisms, including Signals, template event handlers, and `AsyncPipe`, where appropriate.
- Design components to work correctly with zoneless change detection and avoid relying on Zone.js-specific behavior, such as the `NgZone.onMicrotaskEmpty`, `NgZone.onUnstable`, and `NgZone.onStable` observables, which do not emit in zoneless applications.
- Before migrating an existing application, verify compatibility with third-party libraries and existing application code, and run automated tests. Existing applications are not required to migrate immediately.
- Document justified exceptions and migration decisions.
- Refer https://angular.dev/guide/zoneless for further guidance.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

22. Rendering Strategy: CSR, SSR, Prerendering, and Hydration

Angular applications are client-side rendered by default. The rendering strategy defines where and when the HTML of a page is generated. The main terms are:

CSR	Client-Side Rendering: the browser renders the application. The server only needs to serve static assets.
SSR	Server-Side Rendering: the server generates the HTML for each request.
Prerendering	HTML is generated at build time for suitable pages, such as pages that are the same for all users.
Hydration	Angular reuses the server-rendered HTML to initialize client-side interactivity, instead of re-creating the DOM.

- Evaluate the rendering strategy during application architecture design.
- Consider SEO requirements, initial-load performance, content freshness, hosting constraints, security, and operational complexity when selecting a strategy.
- Choose the rendering strategy based on application requirements. SSR is not mandated for every application and does not automatically improve every application.
- Account for the trade-offs: SSR needs a server runtime to render requests, prerendered pages cannot contain user-specific data, and both require code that does not depend on browser-only globals such as `window` and `document`.
- Angular 19 and later can set the rendering mode (client, server, or prerender) per route through the server route configuration (`RenderMode` in `@angular/ssr`).
- For applications using SSR or prerendering, configure hydration where appropriate. Hydration is stable from Angular 17 and is included when SSR is added with the Angular CLI (`ng new --ssr` or `ng add @angular/ssr`). For custom setups, add `provideClientHydration()` and ensure it is also part of the server bootstrap configuration.

```
bootstrapApplication(AppComponent, {
  providers: [provideClientHydration()],
});
```

- Verify that server-rendered and client-rendered output remain consistent. Avoid conditionals that render different content on the server and in the browser (for example `@if` with `isPlatformBrowser`), avoid direct DOM manipulation with native DOM APIs, and use valid HTML structure. Use `ngSkipHydration` only as a temporary, last-resort workaround.
- Document the selected rendering strategy and any relevant trade-offs.
- Refer https://angular.dev/guide/ssr and https://angular.dev/guide/hydration for further guidance.

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved

3. Application Pre-requisite

Hardware Requirements

- Windows 10, 4 GB Ram, 2 CPU, > 1 GB storage
- Linux, 4 GB RAM, 2 CPU, > 1GB storage

Software Requirements

- Node.js – latest stable version (v16)
- Angular – latest stable version (v13)
- VSCode – latest stable version
- Git Client / Git Bash – latest stable version

Browser Support

List of browsers supported by the application

Chrome	Latest
Edge	Latest
Firefox	Latest
Safari	Latest

VS Code extensions

Listing few good to have extensions while working with

- Angular Snippets: Adds angular snippet to quickly create components
- Angular Language Service: provides rich editing to both inline and external templates for your projects.
- ESLint: This extension is a linter for ECMAScript/JavaScript code in your workspace. What it does is identify and report patterns in your code to create a codebase that is more consistent and less buggy
- Prettier: It enforces a consistent style by parsing your code and re-printing it with its own rules that take the maximum line length into account, wrapping code when necessary

© Copyright 2020 WNS (Holdings) Ltd. All rights reserved



CO-CREATE TO OUTPERFORM

This presentation and any files attached and/or transmitted with it are confidential and intended solely for the use of the individual or entity to whom they are addressed. No part of this presentation may be given, lent, resold, or disclosed to any unintended recipients or exploited for any commercial purposes. If you are not the intended recipient and you have received this presentation in error, please return this material to the sender immediately and forthwith delete and destroy the presentation including any copies thereof from your records. We hereby notify that disclosing, distributing, copying, reproducing, storing in a retrieval system, or transmitting in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, or taking any action in reliance on the contents of the presentation in its entirety or any part thereof is strictly prohibited without the prior written consent of WNS, such consent being given at the sole discretion of WNS. Any views or opinion expressed in this presentation are those of the author and do not necessarily represent that of WNS. WNS makes no representations and to the full extent permissible by applicable law, WNS disclaims any warranties of any kind, express or implied, including any warranty of merchantability, accuracy, fitness or applicability for a particular purpose, and non-infringement of third party rights, as to the information, content and materials.

WNS